

SOLUTION DESIGN SPECIFICATION (SDD)

Competitor Quote Comparison Agent

Microsoft Azure & Azure AI Foundry — Solution Design for Henley Homes

Field	Detail
Project	Competitor Quote Comparison Agent — Intelligent Automation
Document	Solution Design Specification (SDD)
Version	0.3 — Draft (Module B design sections added)
Status	Draft
Date	25 May 2026
Prepared by	Drew Alexander, Head of Microsoft AI, Fusion5
Audience	Fusion5 delivery team (Solution Architects, Engineers, Delivery Lead)
Classification	Commercial in Confidence

Document Control

Version History

Version	Date	Author	Summary of change
0.1	25 May 2026	Drew Alexander	Initial draft, derived from the Competitor Quote Comparison Agent FRD (Draft).
0.2	25 May 2026	Drew Alexander	Added Section 3.6 Azure Well-Architected Review (AI workload), scoped to Reliability and Security; added Appendix B open item OI-11 to export the WAR results.
0.3	25 May 2026	Drew Alexander	Added Module B design sections: Responsible AI (14), AI Evaluation / Test & Acceptance (15), Deployment & Release (16), Operational Handover & Support (17), Business Continuity & DR (18), Requirements Traceability & Control Mapping (19).

Reviewers and Approvers

Role	Name	Responsibility	Status
Solution Architect	[Assigned]	Technical correctness and build feasibility	Pending
Subject Matter Expert	Adrian Serratore (Henley)	Pricing logic, ambiguity rules, email format	Pending
Platform / Power Platform	Daniel Pike (Henley)	Environment, DLP, identity, monitoring	Pending
Delivery Lead	[Assigned]	Scope, cost and commercial alignment	Pending

Related Documents

Reference	Document	Owner
FRD	Competitor Quote Comparison Agent — Functional Requirements Document (Draft)	Fusion5 / Henley
SOW / WO	Statement of Work and Work Order for the engagement	Fusion5
Pricing source	Henley internal pricing spreadsheet (state variants)	Henley Marketing

This SDD is a controlled document. The current version is held in the engagement workspace; printed or copied versions are uncontrolled. Material changes after baseline are made through the engagement change-control process.

Contents

Document Control.....	2
1. Introduction.....	6
1.1 Purpose.....	6
1.2 Scope of this Document.....	6
1.3 Intended Audience.....	6
1.4 Definitions and Acronyms.....	6
2. Solution Overview.....	7
2.1 Business Context.....	7
2.2 Problem Statement.....	7
2.3 Solution Summary.....	7
2.4 Scope.....	8
2.5 Design Principles.....	8
3. Solution Architecture.....	9
3.1 Architecture Overview.....	9
3.2 Logical Architecture.....	10
3.3 Technology Stack.....	10
3.4 Why Logic Apps and Durable Functions.....	11
3.5 Environments.....	11
4. Process Design — Happy Path.....	12
4.1 End-to-End Flow.....	12
4.2 Definition of Done.....	13
5. Component Design.....	13
5.1 Intake & Orchestration.....	13
5.2 Document Extraction.....	14
5.3 Comparison Reasoning.....	14
5.4 Human-in-the-Loop.....	14
5.5 Output Generation & Delivery.....	16
5.6 Pricing Data & Document Store.....	17
6. Data & Document Integration Design.....	17
6.1 Integration Surface and Pattern.....	18
6.2 Data Architecture.....	18
6.3 Pricing Data and State Variants.....	18
6.4 Document Lifecycle and Definition of Done.....	18
6.5 Idempotency and Duplicate Prevention.....	19
6.6 Failure Modes.....	19
7. Data and State Model.....	19
7.1 The Process Ledger.....	19
7.2 Process Ledger — Core Fields.....	19
7.3 State Model.....	20
7.4 Document and Data Storage and Retention.....	20
8. Error Handling and Exception Management.....	20
8.1 Failure Philosophy.....	20
8.2 Exception Taxonomy and Escalation.....	20
8.3 Failure Modes by Stage.....	21
8.4 Orphan Prevention.....	21
8.5 The Reconciliation Sweep.....	21

8.6 SLA Clocks and Ageing.....	22
8.7 Requester Visibility.....	22
9. Non-Functional Requirements.....	22
10. Security and Compliance.....	23
10.1 Identity and Access.....	23
10.2 Secrets Management.....	23
10.3 Data Protection.....	23
10.4 Network.....	23
10.5 Audit and Compliance.....	23
11. Monitoring and Observability.....	24
11.1 Telemetry.....	24
11.2 Operational Dashboard.....	24
11.3 Alert Rules.....	24
11.4 Key Performance Indicators.....	25
12. Cost Governance and the Cost Model.....	25
12.1 Cost Model — Basis and Assumptions.....	25
12.2 Unit Rate Reference.....	25
12.3 Per-Request Cost Breakdown.....	26
12.4 Indicative Monthly Cost at Volume.....	26
12.5 Cost Drivers and Optimisation Levers.....	26
12.6 Cost Governance Controls.....	27
13. Assumptions, Dependencies and Risks.....	27
13.1 Assumptions.....	27
13.2 Dependencies.....	27
13.3 Risk Register.....	28
14. Responsible AI.....	28
14.1 Responsible AI Approach.....	28
14.2 Responsible AI Impact Assessment.....	28
14.3 Mapping to the Six Responsible AI Principles.....	29
14.4 Content Safety and Guardrails.....	30
14.5 Grounding and Hallucination Mitigation.....	30
14.6 Human Oversight as a Responsible AI Control.....	30
15. AI Evaluation, Test Strategy and Acceptance.....	30
15.1 Evaluation Approach.....	30
15.2 Evaluation Dataset (Golden Set).....	31
15.3 Evaluation Metrics and Thresholds.....	31
15.4 Automated Evaluation in the Pipeline.....	31
15.5 User Acceptance Testing.....	31
15.6 Acceptance Criteria.....	32
16. Deployment and Release Approach.....	32
16.1 Deployment Strategy.....	32
16.2 CI/CD Pipeline.....	33
16.3 Release Management, Cutover and Rollback.....	33
17. Operational Handover and Support Model.....	33
17.1 Target Operating Model.....	33
17.2 Responsibilities (RACI).....	33
17.3 Service Levels.....	34
17.4 Run Book and Knowledge Transfer.....	34

18. Business Continuity and Disaster Recovery.....34

18.1 Availability and Resilience Design.....34

18.2 Recovery Objectives.....34

18.3 Backup and Recovery.....35

18.4 DR Testing.....35

19. Requirements Traceability and Control Mapping.....35

19.1 Requirements Traceability Matrix.....35

19.2 Module B Control Evidence Map.....36

Appendix A — Glossary.....36

Appendix B — Open Questions.....37

1. Introduction

1.1 Purpose

This Solution Design Specification (SDD) defines the target-state design for the Competitor Quote Comparison Agent, an AI-assisted automation that compares a competitor home-builder quote against Henley Homes' equivalent pricing and produces a customer-ready draft email for the sales team. It translates the agreed functional requirements into an architecture an engineer can build from and a reviewer can assure.

The agent automates work performed today, manually, by a single subject-matter expert in the Henley marketing team. The design therefore optimises for two outcomes that matter more than raw speed: that no comparison ever misleads a customer, and that no request is ever silently lost. Both are treated as first-class design concerns throughout this document, not as afterthoughts.

This is a Fusion5 reference design. Where a value is specific to Henley's environment and must be confirmed during the Design phase, it appears as a bracketed placeholder (for example, [Assigned] or [Tenant]). The open items that must close before the design is baselined are collected in Appendix B.

1.2 Scope of this Document

This document covers the solution architecture, the happy-path process, component design, data and document integration, the data and state model, exception handling, non-functional requirements, security, monitoring, an indicative cost model, and the assumptions, dependencies and risks. It does not restate the functional requirements in full — those are held in the FRD — nor does it constitute a build plan, a test plan or a commercial quotation; the cost model in Section 12 is explicitly indicative and for design purposes only.

1.3 Intended Audience

The primary audience is the Fusion5 delivery team — Solution Architects, Engineers and the Delivery Lead — and so the document is technically precise and assumes fluency with Azure and Azure AI Foundry. It is also written to be assured by Henley stakeholders: Adrian Serratore as the pricing and output authority, and Daniel Pike as the platform owner for environment, identity, data-loss prevention and monitoring.

1.4 Definitions and Acronyms

Key terms are defined where first used and collected in Appendix A. A small number recur often enough to note up front.

Term	Meaning in this document
Comparison / request	One end-to-end run: a competitor quote compared against Henley, ending in a delivered draft email or a named human owner.
SME	Subject-matter expert — Adrian Serratore, the authority on Henley pricing, ambiguity rulings and the email format.
Process ledger	The agent's own record of where every request is; the single source of truth for state (Section 7).

Term	Meaning in this document
Ambiguity flag	A request for human confirmation where a competitor inclusion lacks the detail needed for a reliable comparison (e.g. the downlights case).
FinOps	Cloud financial operations (Section 12) — not Dynamics 365 Finance & Operations.

2. Solution Overview

2.1 Business Context

Henley's sales consultants regularly receive competitor quotes — from builders such as Carlisle Homes and Metricon — from prospective customers who want to understand how Henley compares. Today, each comparison is performed by hand by Adrian Serratore in the marketing team. He identifies the equivalent Henley home range, retrieves the relevant state-specific pricing, compares line items, identifies where competitor inclusions are vague or misleading, surfaces the inclusions Henley provides that the competitor does not, and drafts the email the sales consultant ultimately sends. The work is skilled and high-value, but it is slow, inconsistent in format, and dependent on one person.

Customers are nationwide, with Victoria the primary market and Queensland a growing secondary one, so any comparison must apply the correct regional pricing. The output is not a number on a spreadsheet but a persuasive, accurate, customer-facing email — the artefact that moves a prospect toward a deposit.

2.2 Problem Statement

The efficiency problem is real — Adrian is a bottleneck and a single point of dependency — but it is not the problem this design is most concerned with. The deeper problem is trust. A comparison automation will only be adopted by the sales team if they can rely on it never to produce a misleading comparison and never to lose a request. The failure that destroys that trust is not a visible crash; it is a confidently wrong number sent to a customer, or a request that enters the agent and is never heard from again.

The clearest example, drawn from discovery, is the downlights case: a competitor quote may say “downlights included” without a quantity, while the Henley inclusion is 37 downlights worth \$1,606. A naive line-item comparison of \$0 against \$1,606 overstates Henley's cost and misleads the customer. The agent must detect this class of ambiguity and seek human confirmation before it is allowed to compare such items at all. The design principle that follows is simple: confidently wrong is worse than can't-read.

2.3 Solution Summary

The solution is an event-driven Azure pipeline with an Azure AI Foundry agent at its core and a human-in-the-loop on Microsoft Teams. A sales consultant uploads a competitor quote and the relevant Henley pricing data; the agent extracts and structures the line items, matches the Henley home range and applies state-based pricing, performs a line-by-line comparison, detects and escalates ambiguous or low-confidence items for human confirmation, and finally generates a DRAFT customer-ready email which is returned to the consultant to review and send. It never sends to a customer itself. The logical stages are:

1. Intake and orchestration — capture the uploaded quote and pricing data, stage them, and start a durable, idempotent run.
2. Document extraction — digitise and structure the competitor quote line items with confidence scores.
3. Comparison reasoning — match the Henley range, apply state pricing, compare line items, and detect ambiguity.
4. Human-in-the-loop — resolve low-confidence extractions and confirm ambiguous comparisons on Teams.
5. Output generation — assemble the comparison and savings summary and draft the customer email.
6. Resilience and monitoring — keep every request in a known state, catch anything that goes quiet, and report on it.

2.4 Scope

In scope

- Ingestion of a competitor quote (PDF or image) and the Henley pricing data (Excel/CSV) for one comparison at a time.
- Line-item extraction, home-range matching, and state-based pricing for VIC, NSW and QLD with a nationwide fallback.
- Side-by-side cost comparison, inclusion-differences table, and a separated inclusion/design savings summary.
- Ambiguity and hallucination detection with evidence-backed flags and human confirmation.
- Generation of a DRAFT customer-ready email in Adrian's established format; identification of Henley-advantage inclusions.
- Strict session isolation — each request is processed independently, with no carry-over between runs.

Out of scope

- CRM integration (to be assessed as a future priority).
- Automated sending of the customer email — the output is always a draft for human review.
- Real-time retrieval of competitor pricing from external sources.
- Multi-quote batch processing and comparison of more than one competitor against Henley in a single run.
- Historical comparison trend analysis and any deposit or contract workflow triggers.

2.5 Design Principles

Six principles govern the design. They are the lens through which every later decision should be read.

- **Every request reaches a terminal state.** A run ends either as a delivered draft the consultant can act on, or as a request a named person or team is holding. Nothing ends in “the agent gave up”.
- **Assume routing will sometimes fail.** A reconciliation sweep exists specifically to catch requests that have gone quiet, because the explicit exception paths will not catch everything.

- **The automation owns its state.** A process ledger is the single source of truth for where any request is. “Done” is a verified, delivered draft, not an API status code.
- **Idempotency everywhere.** Every stage is safe to run twice and keys off a stable request identifier, so a retry never produces a duplicate comparison or a duplicate email.
- **The agent proposes; a human disposes.** The agent never invents a price and never finalises an ambiguous comparison. “I can't compare this confidently” is an expected, first-class output, not a failure.
- **Confidently wrong is worse than can't-read.** An item that fails extraction is safe because it escalates. The real danger is a high-confidence but wrong comparison, defended against with cross-field validation and grounded evidence, not confidence thresholds alone.

3. Solution Architecture

3.1 Architecture Overview

The solution is event-driven and message-based. A lightweight intake layer captures the uploaded documents and places a single message on a queue; a durable orchestration then drives the request through extraction, reasoning, optional human steps and output generation, recording every transition to a process ledger. AI services are called as bounded steps within that orchestration rather than being left to run unsupervised. Exceptions and human pauses are explicit branches of the same orchestration, and a separate monitoring and reconciliation layer watches the whole thing.

This shape gives three properties the use case needs: durability (a request survives a transient failure or a multi-hour wait for human input), traceability (the ledger always knows where a request is), and isolation (each request is a discrete orchestration instance with no shared state).

3.2 Logical Architecture

Figure 1 shows the end-to-end flow across six lanes: intake and orchestration, processing, AI extraction and reasoning, the human-in-the-loop, the data and output layer, and resilience and monitoring. Solid arrows are the process flow, dashed red arrows are exception paths, and dashed grey arrows are data calls, retries and telemetry.

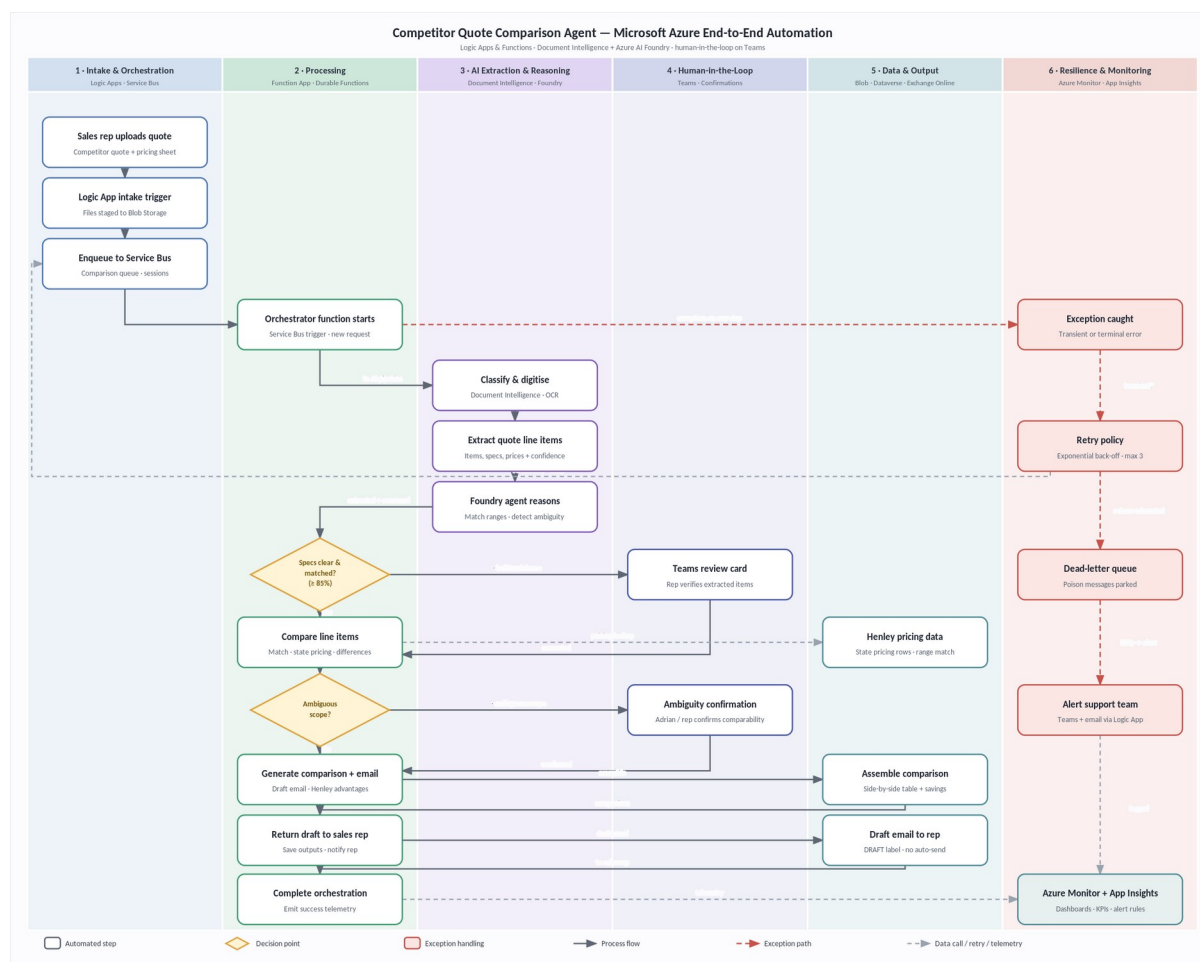


Figure 1 — Competitor Quote Comparison Agent: end-to-end process flow across the six architectural lanes.

3.3 Technology Stack

The solution is built on Azure platform-as-a-service and Azure AI services, with Microsoft Teams and Exchange Online as the human and notification surfaces. No component requires a virtual machine to manage.

Stage / concern	Service	Role in the solution
Intake	Azure Logic Apps (Standard)	Captures the uploaded quote and pricing data, sends the acknowledgement, and enqueues one message per request.
Messaging	Azure Service Bus (Standard)	Decouples intake from processing; sessions and a dead-letter queue give ordered, recoverable delivery.
Orchestration	Azure Functions — Durable Functions	Drives each request as a durable, resumable orchestration that can pause for hours awaiting human input.

Stage / concern	Service	Role in the solution
Document AI	Azure AI Document Intelligence	OCR, classification and structured line-item extraction from the competitor quote, with per-field confidence.
Reasoning	Azure AI Foundry — Agent Service	Matches the Henley range, applies pricing logic, detects ambiguity, and drafts the email; calls tools, never master data.
Human-in-the-loop	Microsoft Teams (Adaptive Cards)	Low-confidence review and ambiguity confirmation, posted to the consultant or to Adrian.
Data & documents	Azure Blob Storage + Dataverse	Blob stages the source documents and outputs; Dataverse holds the process ledger and structured pricing reference.
Notifications	Exchange Online	Request acknowledgement to the consultant and delivery of the DRAFT comparison email.
Monitoring	Azure Monitor + Application Insights	Telemetry, dashboards, alert rules and the data behind the reconciliation sweep.
Secrets & identity	Azure Key Vault + Entra ID	Managed identities and least-privilege access; no secrets in code or configuration.

3.4 Why Logic Apps and Durable Functions

A reasonable question is why the design uses both Logic Apps and Azure Functions rather than one of them. The two solve different problems. Logic Apps Standard is the right tool for intake: it has first-class connectors for file upload, Exchange Online and Service Bus, and it makes the “capture, acknowledge, enqueue” step declarative and easy to change. Durable Functions is the right tool for orchestration: the core of this process is a long-running, stateful workflow that must pause for human input, fan out to AI calls, retry individual steps, and remain idempotent — exactly the orchestration patterns Durable Functions provides in code, with the testability the comparison logic warrants.

Keeping intake thin and declarative while keeping orchestration explicit and testable is a deliberate separation of concerns; it also means a change to the intake channel never touches the comparison logic, and vice versa.

3.5 Environments

Three environments are provisioned — Development, Test/UAT and Production — each in its own resource group with the full tag set described in Section 12.6, and each with an isolated set of AI resources, storage and a Dataverse environment so that no test data can reach a production comparison. UAT is the environment in which Adrian validates the ambiguity logic and the email format against real quotes before go-live. Non-production AI resources use lower capacity tiers and are scaled down out of hours.

3.6 Azure Well-Architected Review (AI Workload)

The design has been assessed against the Azure Well-Architected Framework guidance for AI workloads (the WAR-for-AI), the lens Microsoft recommends for generative-AI solutions alongside the five core pillars. For this engagement the review is scoped to two pillars — Reliability and Security — because they carry the workload's two non-negotiable outcomes: that no request is ever silently lost, and that no comparison ever exposes data or misleads a customer. The findings below

summarise how the target-state design satisfies each pillar and the items to close before go-live. The formal review is run with Henley's platform owner during the Design phase; its exported results are retained as the design assurance artefact and the open items are carried in Appendix B.

Reliability. The design treats every request as a durable, idempotent run tracked in an append-only process ledger, so a failure at any stage leaves the work in a known, recoverable state rather than an orphan (Sections 6–8). A reconciliation sweep catches anything that goes quiet, SLA clocks age every in-flight request, and the human-in-the-loop step degrades gracefully — an ambiguous comparison is escalated for confirmation rather than guessed. Production runs in its own resource group with isolated AI resources (Section 3.5). The reliability considerations to confirm at the review are the model and endpoint availability targets and the failover behaviour for the Foundry agent if a model deployment is throttled or unavailable.

Security. Identity is brokered through Microsoft Entra ID with role-based access control, secrets are held in Key Vault, and data is encrypted at rest and in transit with all traffic over HTTPS (Section 10). The agent never sends to a customer; every output is a draft gated by human review, and the ledger provides an append-only audit trail of who did what to which request. Against the WAR-for-AI security principles the design already addresses content safety, PII minimisation, identity propagation and least-privilege access; the items to confirm at the review are prompt-injection and content-safety filtering on the extraction and reasoning steps, agent-behaviour monitoring for anomalous tool use, and the data-protection classification for the competitor quotes held in the document store.

Pillar	What the design does well	Actions to close before go-live (results exported at review)
Reliability	Durable, idempotent runs; append-only process ledger; reconciliation sweep; SLA ageing; graceful human-in-the-loop degradation; isolated production environment.	Confirm model/endpoint availability targets and Foundry failover under throttling; export Reliability-pillar results.
Security	Entra ID with RBAC; Key Vault secrets; encryption in transit and at rest; draft-only output with mandatory human review; append-only audit trail.	Confirm prompt-injection/content-safety filtering, agent-behaviour monitoring and quote-data classification; export Security-pillar results.

The formal Well-Architected Review against these two pillars is run with Henley's platform owner during the Design phase, and its exported results — the scored assessment and the agreed remediation list — are retained as a design assurance artefact. The open item above is carried in Appendix B until closed.

4. Process Design — Happy Path

4.1 End-to-End Flow

The following describes a successful run in which extraction is confident and no comparison is ambiguous. Each step names the stage that owns it; the exception branches off this path are detailed in Section 8.

7. Intake — a sales consultant uploads the competitor quote and the relevant Henley pricing data, and confirms the customer's region. The Logic App stages both files to Blob Storage, opens a ledger record with a unique request reference, sends the consultant an acknowledgement carrying that reference, and enqueues one Service Bus message.

8. **Orchestration start** — the Durable Functions orchestrator picks up the message, reads the ledger record, and begins the run. The request moves to In Processing.
9. **Extraction** — Document Intelligence digitises the competitor quote and extracts line items (description, specification, price) with per-field confidence. The Henley pricing data is parsed directly from the spreadsheet; it is not sent through Document Intelligence.
10. **Range matching and pricing** — the Foundry agent identifies the closest comparable Henley home range, applies the region's pricing variant, and, where the home-size difference exceeds the agreed tolerance, itemises the cost of the additional squares so the comparison is fair.
11. **Comparison** — the agent maps each competitor line to its Henley equivalent under standard category headings, computes differences, and labels each item Competitor Only, Henley Advantage, or comparable, attaching a confidence indicator to every line.
12. **Savings summary** — the agent totals the differential, separates inclusion savings from design savings, and assembles the inclusion-differences table and the headline Henley value figure, every number traceable to a source line.
13. **Output generation** — the agent drafts the customer-ready email in Adrian's established two-section format, clearly labelled DRAFT, and writes the comparison and the draft to Blob Storage.
14. **Delivery** — the orchestration emails the DRAFT comparison to the consultant for review, marks the ledger record Completed, and emits success telemetry.

4.2 Definition of Done

A request is Done only when the consultant has been delivered a complete, DRAFT comparison email whose every figure is traceable to the source documents, with any flagged items resolved or explicitly noted as discussion points, and the ledger record reflects that delivered state. A 200 response from any intermediate service is not done; a generated email that has not been delivered to the consultant is not done. Anything short of this verified, delivered state remains a live request owned by the agent or by a named human.

5. Component Design

Each logical stage is described below with the same three lead-ins: its responsibility, how it is implemented, and the key design decisions that a reviewer should scrutinise.

5.1 Intake & Orchestration

Responsibility. Capture the uploaded competitor quote and Henley pricing data, acknowledge receipt, create the request's ledger record, and hand a single, idempotent unit of work to the orchestrator.

Implementation. A Logic App (Standard) exposes the upload, stages both files to a per-request Blob container, writes the initial ledger record with a generated request reference, sends the acknowledgement email (Figure 5), and enqueues one Service Bus message keyed by that reference. A Durable Functions orchestrator is triggered from the queue and owns the run thereafter.

Key design decisions. The request reference is the stable business key used by every downstream stage, the ledger, the Teams cards and the emails, so a request is traceable end to end. Intake is deliberately thin: it does no analysis, which keeps the channel easy to change and the comparison

logic isolated from it. The Service Bus message carries only the reference and pointers to the staged files, never the file contents.

5.2 Document Extraction

Responsibility. Turn the competitor quote — a PDF or image of variable format — into structured line items with a confidence score on every field.

Implementation. Azure AI Document Intelligence performs OCR and layout analysis and returns line items (description, specification detail, price) plus document metadata (home name, size, bedrooms, region cues) and per-field confidence. The Henley pricing data is parsed directly from the supplied spreadsheet into structured rows; it never passes through Document Intelligence, since it is already structured and is the pricing reference, not an extraction target.

Key design decisions. Extraction confidence is captured per field, not just per document, because the human review in Section 5.4 corrects individual low-confidence items rather than the whole quote. A custom extraction model trained on representative competitor quotes is the accuracy target; a prebuilt layout model with agent-side extraction is retained as a cheaper fallback to be validated against evaluation data (see Section 12.5).

5.3 Comparison Reasoning

Responsibility. Match the Henley range, apply the correct state pricing, compare every line item, detect ambiguity and hallucination risk, and assign a confidence indicator to each comparison — without ever fabricating a figure.

Implementation. An Azure AI Foundry agent orchestrates the reasoning as a bounded set of tool calls: a range-matching tool, a pricing-lookup tool over the structured Henley data, and a comparison tool that computes differences. The agent is given the extracted competitor lines and a candidate set of Henley rows — never the entire catalogue — and is instructed to ground every figure in a supplied source line or mark it Unconfirmed.

Key design decisions. Three guards defend against confidently-wrong output. First, no figure may appear that is not traceable to a source line; un-sourced numbers are flagged, never estimated. Second, an ambiguity scan detects competitor inclusions stated without quantity or scope (“downlights included”, “driveway included”) and items that look like optional “nice-to-haves”, and routes them to human confirmation before they can be compared. Third, every line carries a HIGH / MEDIUM / LOW confidence indicator, and all LOW and MEDIUM items are summarised with the evidence used or missing. The confidence threshold for automatic acceptance is a tunable parameter, confirmed in the Design phase.

5.4 Human-in-the-Loop

Responsibility. Resolve the two situations the agent must not decide alone: a line item extracted with low confidence, and a comparison whose scope is ambiguous. The orchestration pauses until a human responds.

Implementation. Durable Functions raises an external event and posts an Adaptive Card to Microsoft Teams. The low-confidence review card (Figure 2) shows the extracted items, highlights and makes

editable those below the confidence threshold, and always links the source quote. The ambiguity confirmation card (Figure 3) states the discrepancy plainly — the competitor wording against the Henley inclusion — and offers the reviewer a clear choice. On response, the orchestration resumes from where it paused.

The two cards are deliberately routed differently. A low-confidence extraction is a data-quality fix and goes to the consultant who raised the request; an ambiguity ruling is a judgement call on Henley's pricing and goes to Adrian (or a delegate). Both decisions are written to the ledger history.

WIREFRAME · MICROSOFT TEAMS · LOW-CONFIDENCE EXTRACTION REVIEW

QC

Quote Comparison Agent · just now

Adaptive card · the workflow is paused awaiting this reviewer

1 Quote needs a quick check LOW CONFIDENCE

2 QC-7841 · Carlisle Homes · ledger QCP-2026-7841

3 Carlisle-Brighton-quote.pdf — open source document

EXTRACTED LINE ITEMS

Competitor

Carlisle Homes

Home / range

Brighton 28 — 4 bed, 2 storey

Quote total

\$398,500

Downlights (line)

Included — qty unspecified !

Stone benchtop

20mm included

Driveway (line)

Included — scope unclear !

! Highlighted items fell below the confidence threshold — please review and correct.

6 Confirm & continue

Open source document

Can't resolve — escalate

HOW THE REVIEWER USES THIS

1 Posted to the sales rep (or Adrian) in Teams. The Durable Functions orchestration is paused, awaiting their response within the review SLA.

2 Quote, competitor and ledger reference — the same keys used throughout, so the comparison stays traceable end to end.

3 The source quote is one click away. The reviewer always works from the original PDF, never from extracted data alone.

4 Items extracted with high confidence are shown for context and confirmed at a glance.

5 Low-confidence items are highlighted and editable — the reviewer corrects only what needs attention.

6 "Confirm & continue" returns the corrected line items and the orchestration resumes from where it paused. "Can't resolve" escalates the comparison as an exception.

Reference design — element layout and labels are indicative; the production card is built in the Design phase. Confidence threshold and SLA per Sections 5.3 and 8.6.

Figure 2 — Teams low-confidence extraction review: the consultant verifies and corrects only the items below the confidence threshold.

WIREFRAME • MICROSOFT TEAMS • AMBIGUITY CONFIRMATION

Quote Comparison Agent • 9 min ago
Confirmation task - routed to Adrian (SME) or the sales rep

Confirm comparison — downlights AMBIGUITY FLAG

QC-7841 • Carlisle Homes • ledger QCP-2026-7841

Carlisle states "downlights included" — quantity unspecified.
Henley includes 37 (\$1,606). A direct comparison may mislead.

ITEM DETAIL

Category	Internal Electrical Pack
Competitor states	Downlights included (qty —)
Henley standard	14 downlights + 2-way switches
Henley full pack	49+ downlights • \$1,606

COMPARISON SO FAR

42 of 44 items compared • 2 flagged for confirmation view

ADD A NOTE (recorded to the comparison history)

Treat as comparable Flag as discussion point

Provide competitor quantity LA 4h • 3h 11m left

HOW THE REVIEWER USES THIS

- 1 Routed to Adrian (SME) or the rep — a judgement call on whether two items are genuinely comparable, not a data-quality fix.
- 2 The ambiguity is stated plainly, with the competitor wording and the Henley inclusion side by side — no hunting.
- 3 The rest of the comparison is already built. The reviewer decides only this flagged item — nothing else.
- 4 The decision returns to the workflow. "Provide quantity" lets the reviewer enter the true competitor count instead.
- 5 The note is written to the process ledger History — it becomes part of the comparison's audit trail.
- 6 An SLA clock is running. If it elapses, the comparison escalates rather than waiting silently (Section 8.6).

Reference design — element layout and labels are indicative; the production card is built in the Design phase. Exception routing per Section 8.2.

Figure 3 — Teams ambiguity confirmation: the reviewer rules on whether two items are genuinely comparable (the downlights case).

Key design decisions. Each card states only what the reviewer needs to decide and nothing else, so the rest of the comparison is already assembled and waiting. Every human task carries an SLA clock (Section 8.6); if it elapses, the request escalates rather than waiting silently. "Can't resolve" and "Flag as discussion point" are always available, so a reviewer is never forced into a false choice.

5.5 Output Generation & Delivery

Responsibility. Assemble the final comparison and savings summary and draft the customer-ready email in Adrian's established format, then deliver it to the consultant — never to the customer.

Implementation. The agent generates the email body following the reference two-section structure (inclusions and design), with sub-bullets, per-item savings callouts, a headline value figure, and any LOW/MEDIUM items framed as discussion points rather than stated facts. The email is labelled DRAFT, written to Blob Storage with the structured comparison, and emailed to the consultant (Figure 4).

Delivery is an independently retryable step: a failed send is retried without re-running the comparison or re-creating the draft, because the orchestration keys off the request reference and the draft already exists in storage.

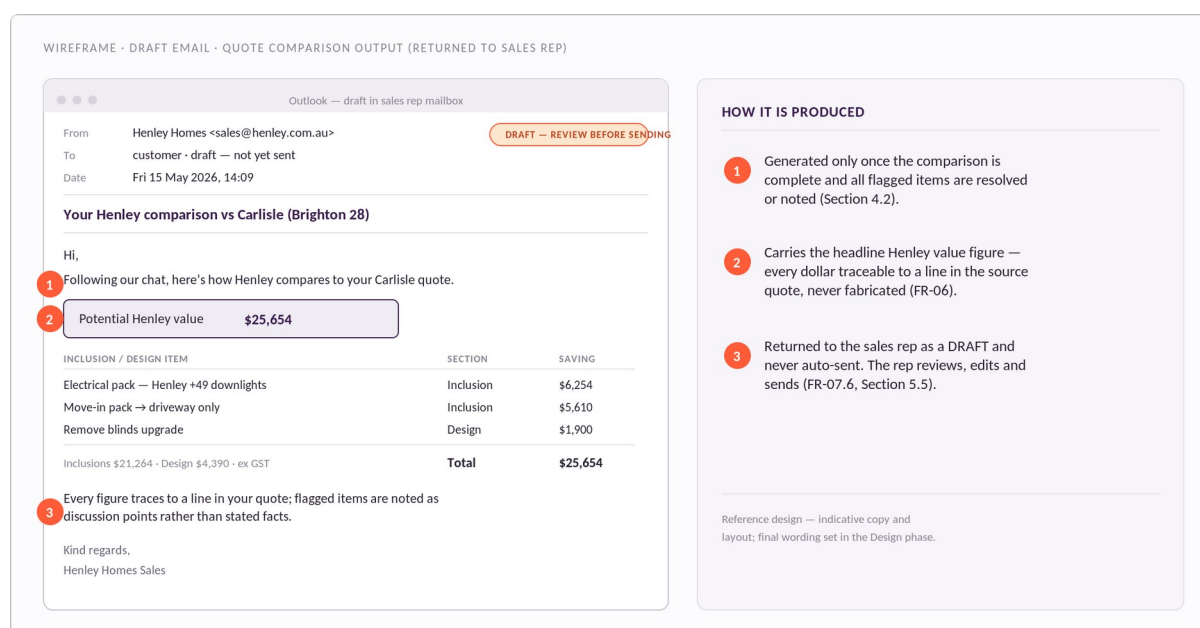


Figure 4 — The DRAFT comparison email returned to the sales consultant: headline value, savings breakdown, and a DRAFT banner.

Key design decisions. The agent has no send-to-customer capability at all — the constraint is structural, not a setting — so an accidental auto-send is impossible. The email avoids internal jargon and spreadsheet references, and the DRAFT label and “review before sending” instruction are mandatory on every output.

5.6 Pricing Data & Document Store

Responsibility. Hold the source documents, the structured Henley pricing reference, and the generated outputs, with isolation between requests.

Implementation. Blob Storage holds the uploaded quote, the pricing snapshot used for the run, and the generated comparison and draft, all under a per-request prefix. The Henley pricing spreadsheet is parsed into structured pricing rows (per region) used by the pricing-lookup tool. The process ledger lives in Dataverse (Section 7).

Key design decisions. The pricing snapshot used for a run is retained with that run, so a comparison can always be explained against the exact data it used even after the master spreadsheet is updated. Per-request prefixes enforce the session isolation required by the FRD: no request can read another request's documents.

6. Data & Document Integration Design

This solution has no transactional ERP as its system of record. Its authoritative inputs are documents — the competitor quote and the Henley pricing data — and its system of record for pricing is the Henley pricing spreadsheet, maintained by the marketing team. This section describes how those inputs are received, stored and structured, and how the design defends data integrity and

idempotency. A future CRM integration is explicitly out of scope (Section 2.4) and is noted only where it affects the data design.

6.1 Integration Surface and Pattern

There are three integration surfaces: file ingestion at intake (the quote and the pricing data), the structured pricing reference the agent queries, and the outbound notification surface (Exchange Online) for the acknowledgement and the draft email. All ingestion is pull-based and request-scoped: nothing is read or written outside the boundary of a single request. The pattern is document-staging plus a structured reference store, rather than a live API integration to a line-of-business system.

6.2 Data Architecture

Three stores carry the data, each chosen for a specific need. Blob Storage holds unstructured source documents and generated outputs; a structured pricing reference (derived from the Henley spreadsheet) backs the pricing-lookup tool; and Dataverse holds the process ledger and the request metadata. The decision table below records why each data need is met the way it is.

Data need	Mechanism	Why
Source quote (PDF/image)	Blob Storage, per-request prefix	Unstructured, large, must be retained with the run for traceability and re-review.
Henley pricing data	Parsed to structured rows + snapshot in Blob	Spreadsheet is the pricing system of record; a per-run snapshot lets any comparison be explained against the exact data used.
Generated comparison + draft	Blob Storage, per-request prefix	Outputs are documents; storing them makes delivery independently retryable without re-running the comparison.
Request state & metadata	Dataverse (process ledger)	Queryable, governed, and the basis for the dashboard, SLA clocks and the reconciliation sweep.
Human decisions / history	Dataverse, on the ledger record	Decisions must be auditable and tied to the request; Dataverse gives row-level security and audit.

6.3 Pricing Data and State Variants

The Henley pricing spreadsheet carries state-specific variants. The agent applies the variant for the customer's confirmed region — VIC (primary), QLD (secondary) and NSW — with a nationwide fallback only where a state is undetermined, and always surfaces the applied variant in the output. Incorrect state pricing is treated as a critical failure (Section 9). The definitive list of state variants, and whether the spreadsheet is a single multi-tab workbook or several files, are open items for Adrian to confirm (Appendix B, OI-01 and OI-03).

6.4 Document Lifecycle and Definition of Done

A document's lifecycle is: staged at intake, snapshotted (pricing) or extracted (quote), referenced during reasoning, and finally superseded by the generated outputs. The integration definition of done mirrors Section 4.2: the run is complete only when the generated draft has been delivered to the consultant and the ledger reflects it. Storage retention is covered in Section 7.4.

6.5 Idempotency and Duplicate Prevention

Every stage keys off the request reference and is safe to run twice. Re-processing the same request — whether from a Service Bus redelivery or a manual replay — reads the existing ledger record and the already-staged documents and converges on the same outputs rather than creating new ones. The acknowledgement carries the reference back to the consultant precisely so that a re-submission of the same quote can be recognised and de-duplicated at intake rather than spawning a second comparison.

6.6 Failure Modes

Area	Failure mode	Design response
Quote ingestion	Unreadable or corrupt PDF/image	Extraction fails closed; request escalates as a low-confidence/technical exception, never guessed.
Pricing data	Stale or missing state variant	Run pauses for confirmation; the applied variant is always shown; currency is the marketing team's responsibility (assumption A-1).
Range matching	No Henley range within tolerance	Flagged to the consultant for confirmation rather than forcing a poor match.
Output delivery	Email send fails	Independently retried off the stored draft; no re-run, no duplicate.
Duplicate submission	Same quote uploaded twice	Reference-based de-duplication at intake; second submission links to the existing request.

7. Data and State Model

7.1 The Process Ledger

The process ledger is the agent's own record of every request and the single source of truth for where each one is. It exists because no external system in this solution can answer the question “where is request X and who owns it?” — there is no ERP order to query. The ledger is written at every state transition and is the data behind the dashboard, the SLA clocks and the reconciliation sweep. It is the mechanism by which the design guarantees that no request is ever silently lost.

7.2 Process Ledger — Core Fields

Field	Purpose
Request reference	Stable business key; ties documents, Teams cards, emails and telemetry together.
Consultant / requester	Who raised the request and receives the acknowledgement and the draft.
Competitor & home range	Extracted metadata: competitor name, home name, size, bedrooms.
Region / pricing variant	The confirmed state and the pricing variant applied.
State	Current lifecycle state (Section 7.3).
Confidence summary	Count of HIGH/MEDIUM/LOW items and any open flags.
Owner	The person or team currently holding the request when it is not with the agent.
SLA due	When the current non-terminal state must be actioned by.

Field	Purpose
Document pointers	Blob references to the source quote, pricing snapshot, comparison and draft.
History	Append-only log of transitions and human decisions, for audit.

7.3 State Model

Every request occupies exactly one state. Every non-terminal state carries an SLA, and only the terminal states end a request's life.

State	Meaning	Terminal?
Received	Intake complete, acknowledged, queued.	No
In Processing	Orchestration running extraction and comparison.	No
Awaiting Review	Paused for a low-confidence extraction fix (consultant).	No
Awaiting Confirmation	Paused for an ambiguity ruling (Adrian/SME).	No
Exception	A business or technical exception is being handled.	No
Manual Fallback	Handed to a person to complete outside the agent.	No
Completed	DRAFT comparison delivered to the consultant and verified.	Yes
Closed — Not Actionable	Not a valid quote, or withdrawn; recorded with reason.	Yes

7.4 Document and Data Storage and Retention

Source documents, the pricing snapshot and the generated outputs are retained per request in Blob Storage for a defined retention period (default [retention period], to be confirmed), then deleted by lifecycle policy. The ledger record and its history are retained longer for audit and KPI reporting. All stores are encrypted at rest. Because customer quotes may contain personal information, retention and deletion are aligned to Henley's data-handling policy (Appendix B, OI-07).

8. Error Handling and Exception Management

8.1 Failure Philosophy

This is the heart of the design. The governing rule is that a request may slow down, escalate or hand off, but it may never disappear, and the agent may never present an unverified comparison as fact. Business exceptions — matters of judgement or missing information — go to people; technical exceptions — transient faults — are retried automatically; the two are never conflated. Every path, including every failure path, ends in either a delivered draft or a named human owner.

8.2 Exception Taxonomy and Escalation

Class	Example	Routed to	Mechanism
Low-confidence extraction	A line item below the confidence threshold.	Consultant (requester)	Teams review card; orchestration pauses (Fig. 2).
Ambiguous comparison	"Downlights included", quantity unspecified.	Adrian / SME	Teams confirmation card; pauses (Fig. 3).

Class	Example	Routed to	Mechanism
No Henley match	No range within size tolerance.	Consultant	Confirmation prompt; manual range selection.
Not a quote	Uploaded file is not a competitor quote.	Consultant	Closed — Not Actionable, with reason.
Technical — transient	Throttled AI call, timeout.	Automatic	Retry with exponential back-off, max 3 (Section 8.3).
Technical — terminal	Repeated failure, poison message.	Support team	Dead-letter queue → alert (Section 8.3).

8.3 Failure Modes by Stage

Stage	Failure modes	Orphan risk & design response
Intake	Upload incomplete; acknowledgement send fails.	Ledger record created before acknowledgement, so the request exists even if the email fails; send is retried.
Extraction	Unreadable document; low confidence.	Fails closed and escalates; never guesses. Request held in Awaiting Review.
Reasoning	Throttling/timeout; ambiguous scope.	Transient faults retried (max 3); ambiguity escalates to confirmation. Orchestration state is durable across the wait.
Human step	Reviewer never responds.	SLA clock elapses → escalation, not silent wait (Section 8.6).
Delivery	Email send fails.	Independently retried off the stored draft; ledger stays In Processing until delivery verified.
Anywhere	Unhandled error; lost message.	Dead-letter queue + reconciliation sweep catch what explicit paths miss (Sections 8.4–8.5).

8.4 Orphan Prevention

An orphan is a request that has entered the system but no longer has an owner, a next action or visibility — the single failure that would destroy trust in the agent. The design prevents orphans structurally: a ledger record is created at intake before anything else can fail; every state transition is written to the ledger; every non-terminal state has an SLA and an owner; and a reconciliation sweep exists specifically to catch anything the explicit paths miss.

8.5 The Reconciliation Sweep

On a schedule, a reconciliation function queries the ledger for any request in a non-terminal state past its SLA, or with no recorded transition within an expected window, and forces it onto an exception path with a named owner. The sweep assumes the explicit exception handling will sometimes fail — a message lost, an event never delivered — and is the safety net that makes the no-orphan guarantee real rather than aspirational. Anything it surfaces appears on the at-risk panel of the operational dashboard (Section 11.2).

8.6 SLA Clocks and Ageing

Each non-terminal state carries an SLA appropriate to it — short for automated steps, longer for human ones (a default human-review SLA of [SLA] is assumed, to be confirmed). The clock is recorded on the ledger when the state is entered. On elapse, the request escalates: a reminder, then reassignment or an exception, so a request waiting on a person who is unavailable does not sit silently. Ageing is visible on the dashboard so that human capacity becoming a bottleneck is seen early.

8.7 Requester Visibility

The consultant who raised a request is kept informed without having to chase. An acknowledgement is sent the moment the quote is uploaded, carrying the request reference (Figure 5); the draft is delivered on completion; and if a request needs the consultant's input, the Teams card is itself the notification. The reference lets the business correlate any follow-up to the request and is the basis for de-duplicating a re-submission.

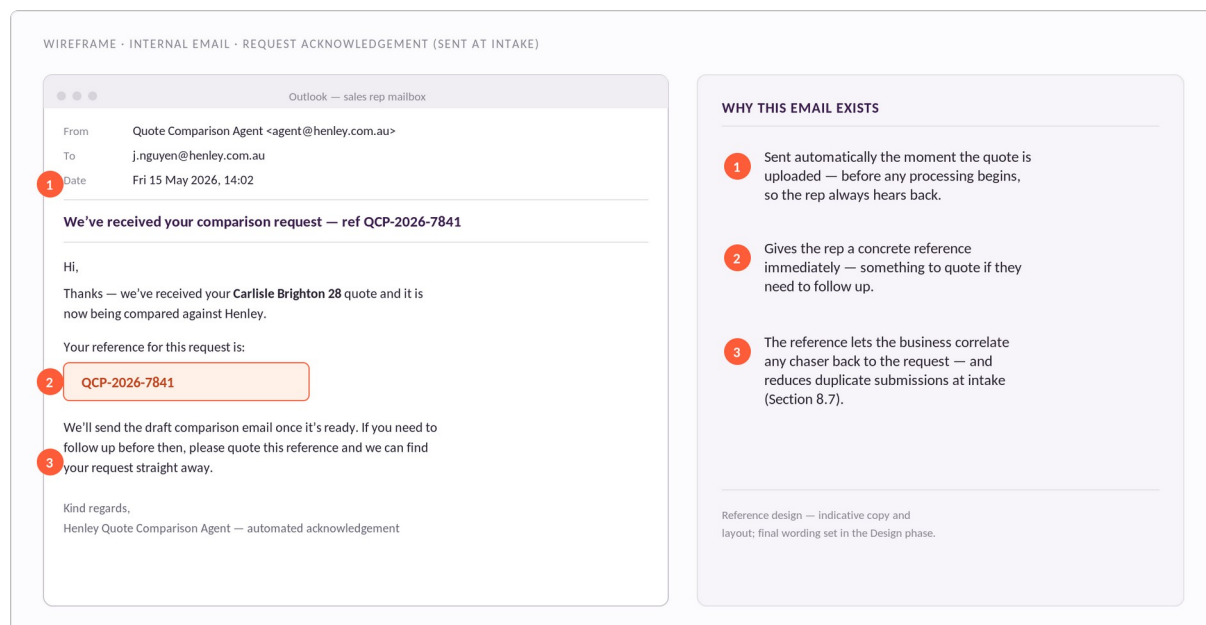


Figure 5 — Request acknowledgement sent at intake, carrying the reference the consultant can quote to follow up.

9. Non-Functional Requirements

The targets below are the design's working assumptions; volumetrics are confirmed in the Design phase against Henley's actual demand.

Category	Requirement	Target / approach
Accuracy	No fabricated cost, quantity or inclusion.	Zero tolerance; every figure traceable to a source line, or flagged Unconfirmed (Section 5.3).
Session isolation	Each request fully independent.	Per-request orchestration instance, Blob prefix and pricing snapshot; no shared state.
Throughput	Handle the sales team's daily demand with headroom.	Queue-based; scales horizontally with Functions; modelled to thousands/month (Section 12).
Peak handling	Absorb bursts without loss.	Service Bus buffers; back-pressure rather than dropped requests.

Category	Requirement	Target / approach
Latency	Draft ready in minutes for a clean run.	Bounded AI calls; human steps excluded from the automated-latency target.
Availability	Available in Henley business hours across states.	PaaS services with platform SLAs; no single VM dependency.
Recoverability	Survive transient faults and restarts.	Durable orchestration state; idempotent, resumable stages.
Maintainability	Pricing and prompt logic changeable safely.	Pricing data externalised; prompts and thresholds versioned and evaluated.
Data integrity	Correct state pricing always applied.	State variant surfaced in output; incorrect state pricing is a critical failure.

10. Security and Compliance

10.1 Identity and Access

All service-to-service authentication uses Entra ID managed identities; there are no shared keys in code or configuration. Each component runs under a least-privilege identity — the extraction function can read a request's blobs but not write the ledger's audit history, and so on. Human access to Teams cards is governed by Entra ID group membership, so only authorised consultants and Adrian (or a delegate) can act on a request.

10.2 Secrets Management

Any remaining secrets — connection strings, AI service endpoints where a managed identity is not available — are held in Azure Key Vault and referenced at runtime, never embedded. Access to Key Vault is itself granted by managed identity and audited.

10.3 Data Protection

Competitor quotes and the resulting comparisons may contain personal information about a prospective customer. All data is encrypted in transit and at rest; documents are stored per request and deleted on the retention schedule (Section 7.4); and the design holds only what a comparison needs. Data residency is Australia East (Section 12.1), to be confirmed against Henley's policy.

10.4 Network

Components communicate over Azure's backbone; public exposure is limited to the intake and Teams surfaces. Private endpoints for storage, Dataverse and the AI services, and the network posture overall (including any DLP requirements Daniel Pike specifies), are confirmed in the Design phase.

10.5 Audit and Compliance

The ledger history is an append-only audit trail of every state transition and every human decision, tied to the request reference and the acting identity. Together with Azure platform logs, it provides a complete, queryable record of who did what to which request and when — sufficient for internal assurance and for demonstrating that no comparison was sent without human review.

11. Monitoring and Observability

11.1 Telemetry

Every stage emits structured telemetry to Application Insights, correlated by the request reference, so a single request can be traced across intake, extraction, reasoning, human steps and delivery. Ledger transitions are the business-level telemetry; Azure platform metrics are the infrastructure-level telemetry. The two together answer both “is the platform healthy?” and “is work flowing?”.

11.2 Operational Dashboard

A single operational dashboard, built over the process ledger, gives the operations team the health of the agent at a glance: headline KPIs, every in-flight request by state, the orphan count, the exception mix, the human-review queue depth and ageing, and a table of requests past SLA surfaced by the reconciliation sweep (Figure 6).

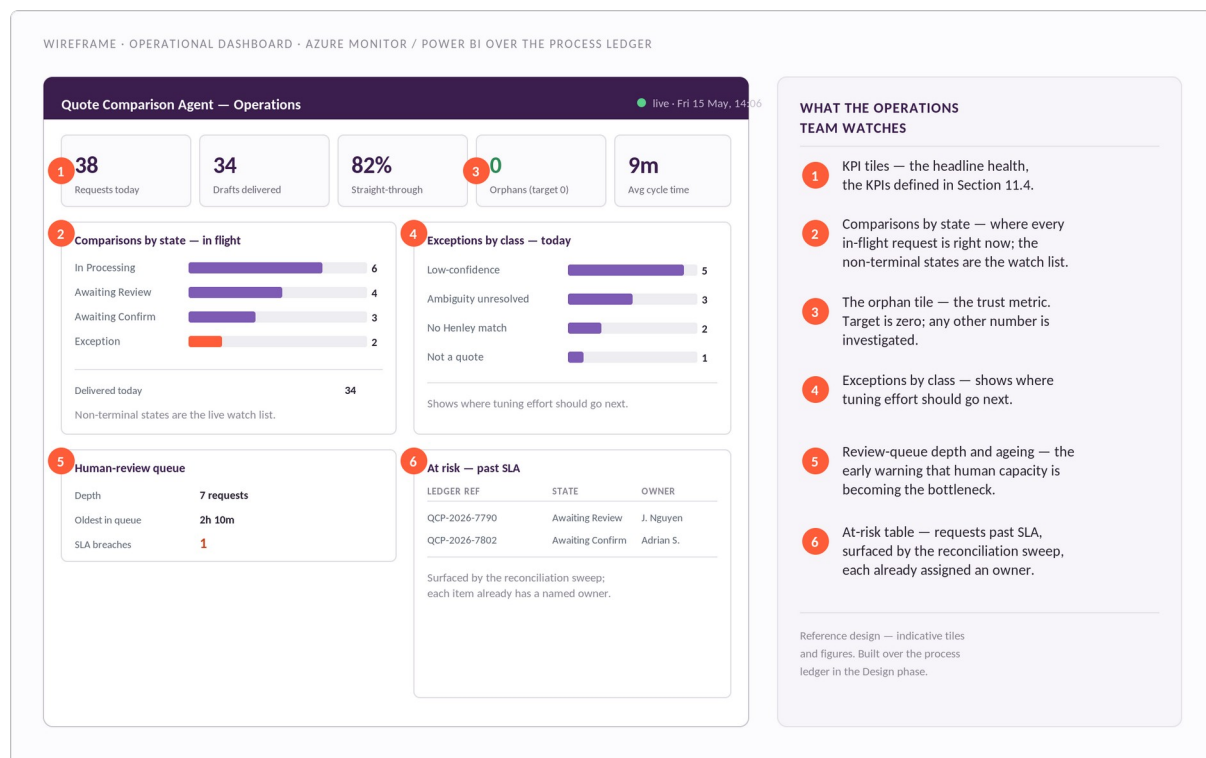


Figure 6 — Operational dashboard over the process ledger: KPI tiles, states in flight, exception mix, review queue and at-risk requests.

11.3 Alert Rules

Alert	Condition	Action
Orphan detected	Reconciliation sweep finds a request with no owner.	High-priority alert to support; auto-assign to a fallback owner.
SLA breach	A human step exceeds its SLA.	Reminder, then reassignment/escalation.
Dead-letter growth	Messages accumulating in the DLQ.	Alert support; investigate poison message.
AI error/throttle spike	Error or throttle rate over threshold.	Alert; the volume-sensitive cost line is also watched (Section 12.6).

Alert	Condition	Action
Extraction confidence drop	Sustained fall in average confidence.	Alert — likely a new competitor quote format; review extraction model.

11.4 Key Performance Indicators

KPI	What it tells us
Straight-through rate	Share of requests completed with no human step — the primary efficiency and cost lever.
Cycle time	Time from upload to delivered draft (automated portion).
Exception rate by class	Where tuning effort should go next.
Orphan count	The trust metric; target is zero.
SLA adherence	Whether human steps are actioned in time.
Human-review turnaround	How quickly reviewers respond; early warning of a capacity bottleneck.

12. Cost Governance and the Cost Model

This section gives an indicative cost model for design — what the solution costs to run, what drives that cost, and how cost is governed. It is not a quotation. Figures are built bottom-up from current Azure pay-as-you-go list prices and converted to Australian dollars; they exclude GST and any Enterprise Agreement or MCA discount, and should be confirmed against the Azure Pricing Calculator before any commercial commitment.

12.1 Cost Model — Basis and Assumptions

Pricing basis: Australia East, Australian dollars, pay-as-you-go, researched May 2026. Working FX rate: US\$1.00 = A\$1.40 (i.e. A\$1.00 ≈ US\$0.71); list prices are USD and converted at this stated rate. Document profile: an average competitor quote of ~8 pages; the Henley pricing data is parsed directly and incurs no document-AI cost. Agent profile: one run per request, with an estimated ~50,000 input and ~10,000 output tokens including tool round-trips. Scope: Azure consumption only — human-review labour is excluded and is governed by the straight-through rate. Volume is modelled at 250, 1,000 and 5,000 requests per month so that both unit economics and fixed-cost dilution are visible; realistic early demand is expected in the low hundreds per month.

12.2 Unit Rate Reference

Service	Meter	Indicative rate (AUD)
Document Intelligence	Custom extraction, per 1,000 pages	≈ A\$42 (US\$30); prebuilt layout ≈ A\$14 (US\$10)
Model tokens — GPT-4.1	Input / output per 1M tokens	≈ A\$2.80 in / A\$11.20 out
Model tokens — GPT-4.1-mini	Input / output per 1M tokens	≈ A\$0.56 in / A\$2.24 out
Azure AI Foundry — Agent Service	Service charge	No charge for the service; pay tokens + tools

Service	Meter	Indicative rate (AUD)
Logic Apps (Standard)	WS1 hosting plan, per month	≈ A\$245 (US\$175)
Azure Functions (Flex)	Per-million executions + GB-seconds	Rounding error at these volumes
Service Bus (Standard)	Base charge per month	≈ A\$14 (US\$10) + operations
Dataverse	Storage per GB/month (overage)	≈ A\$56/GB; minimal for the ledger
Application Insights	Ingestion per GB (first 5 GB free)	≈ A\$3.20/GB
Blob Storage, Key Vault	Per GB/month; per 10k operations	Rounding error

12.3 Per-Request Cost Breakdown

Two costs dominate and scale with volume: the document-AI extraction and the agent's model tokens. The table shows the variable cost of one ~8-page comparison for a capable model and for a “mini”.

Cost component	GPT-4.1 (AUD)	GPT-4.1-mini (AUD)
Document extraction (~8 pages, custom)	≈ A\$0.34	≈ A\$0.34
Model tokens (~50k in / 10k out)	≈ A\$0.25	≈ A\$0.05
Messaging / functions / storage	≈ A\$0.01	≈ A\$0.01
Variable cost per request	≈ A\$0.60	≈ A\$0.40

12.4 Indicative Monthly Cost at Volume

Fixed platform cost is dominated by the Logic Apps Standard plan; everything else is small. The effective cost per request falls as volume rises through fixed-cost dilution. Figures use the GPT-4.1 variable cost and a fixed platform floor of ≈ A\$300/month.

Volume / month	Variable	Fixed platform	Total / month	Effective / request
250	≈ A\$150	≈ A\$300	≈ A\$450	≈ A\$1.80
1,000	≈ A\$600	≈ A\$300	≈ A\$900	≈ A\$0.90
5,000	≈ A\$3,000	≈ A\$300	≈ A\$3,300	≈ A\$0.66

With a mini model the variable cost falls by roughly a third again; at 1,000/month the total is ≈ A\$700 (≈ A\$0.70/request). At these volumes the dominant cost is the fixed Logic Apps plan, which is the first thing to revisit for a low-volume internal tool (Section 12.5).

12.5 Cost Drivers and Optimisation Levers

- **Straight-through rate is the real lever.** Every request that needs a human carries a labour cost far larger than its Azure cost. Raising the straight-through rate — by tuning extraction and ambiguity detection — is the biggest cost action, and it is tuning, not infrastructure (cross-reference Section 11.4).
- **Document-AI model choice.** Prebuilt layout extraction is roughly 3x cheaper per page than custom; validate accuracy against evaluation data before adopting it.
- **Language-model selection.** A mini model is ~5x cheaper per token and is often sufficient for grounded matching and completeness checks; reserve the capable model for the email drafting.

- **Fixed-cost floor.** At low volume the Logic Apps Standard plan dominates; a Consumption-based intake is a cheaper option to evaluate for a low-volume internal tool.
- **Token discipline.** Tight prompts, passing candidate Henley rows rather than the whole pricing set, and capping agentic loop iterations keep token cost down.
- **Telemetry and non-prod hygiene.** Sample Application Insights at volume and scale non-production resources down out of hours.

12.6 Cost Governance Controls

Every resource sits in a per-environment resource group with a mandatory tag set enforced by Azure Policy: Solution, Environment, CostCentre, Owner and DataClassification. Azure Cost Management budgets are set per environment with alerts at 50/80/100% of forecast, plus a specific spend alert on the AI resources because they are the volume-sensitive lines most exposed to a prompt regression or a change in competitor quote format. Cost is reported monthly against the tags (showback), with effective cost per request tracked alongside the KPIs. A monthly actual-versus-modelled review, owned by the Delivery Lead and handed to the Henley owner at transition, revisits these levers as real volume data accrues.

13. Assumptions, Dependencies and Risks

13.1 Assumptions

- A-1 — The Henley pricing spreadsheet is maintained and updated by the marketing team before each use; the agent is not responsible for pricing currency.
- A-2 — Competitor quotes are provided as PDF or image files; the agent performs OCR/extraction from these formats.
- A-3 — The agent is hosted in a secure Azure environment accessible only to Henley staff.
- A-4 — Adrian Serratore is available during UAT to validate ambiguity logic and the email format, and is the standing authority for ambiguity rulings in production.
- A-5 — The Panama 255-S27 email is the authoritative reference for the output format unless updated by Adrian.

13.2 Dependencies

- D-1 — Azure tenant, subscription and the three environments provisioned by Henley platform/IT (Daniel Pike).
- D-2 — Access to a representative sample of competitor quotes to train and evaluate the extraction model.
- D-3 — The definitive Henley pricing data and its state-variant structure, confirmed by Adrian.
- D-4 — Microsoft Teams and Exchange Online available for the human and notification surfaces.

13.3 Risk Register

Risk	L	I	Mitigation	Owner
Misleading comparison reaches a customer	Low	High	Ambiguity detection + human confirmation; DRAFT-only output; UAT validation with Adrian.	[Assigned]
Variable competitor quote formats reduce extraction accuracy	Med	Med	Custom model on representative samples; low-confidence escalation; confidence-drop alert.	[Assigned]
Incorrect state pricing applied	Low	High	Variant surfaced in output; treated as critical failure; tested per region in UAT.	[Assigned]
Pricing spreadsheet stale or inconsistent	Med	Med	Per-run snapshot; currency is a marketing responsibility (A-1); pause on missing variant.	Henley Marketing
Adoption stalls if sales team distrust output	Med	High	Transparency (confidence flags, traceable figures); early demo; Adrian as visible owner.	[Assigned]
Low volume makes fixed cost disproportionate	Med	Low	Evaluate Consumption intake; mini model; monthly cost review.	Delivery Lead

14. Responsible AI

14.1 Responsible AI Approach

This solution is designed and will be operated in line with the Microsoft Responsible AI Standard and its six principles — fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability. Responsible AI is treated here as a design input, not a post-hoc review: the controls below shape where content safety sits, how the agent is grounded, and where the accountable human acts. The agent makes no autonomous decision that reaches a customer; every external-facing output is reviewed and released by a Henley sales consultant, which is the primary accountability control for the whole solution.

A named Responsible AI owner (Fusion5 AI Practice Lead, with a Henley business owner counterpart) is accountable for sign-off on this section before the design is baselined and for re-confirming it at each material model or prompt change. The RAI Impact Assessment in 14.2 is the living record of that review.

14.2 Responsible AI Impact Assessment

The intended use is narrow and well-bounded: compare a single competitor home-builder quote against Henley's equivalent pricing and draft an internal-to-customer email for a consultant to review. The agent does not set prices, does not contact customers directly, and does not make eligibility or credit decisions about people. The assessment below records the foreseeable harms for this intended use and the design controls that mitigate them.

Potential harm / failure	Severity	Design mitigation
Inaccurate comparison — wrong line items matched or a price misstated, leading to a misleading claim to a customer	High	Numeric values are read from the authoritative Henley pricing store, not generated by the model (14.5); comparison logic is deterministic where possible; groundedness is evaluated (Section 15); a consultant reviews every draft before send (14.6).
Hallucinated content — invented inclusions, prices or competitor terms	High	“Insufficient evidence escalates, it does not guess” (Section 8 failure philosophy); grounding and groundedness-detection; low-confidence extractions route to human review rather than into the draft.
Indirect prompt injection — a competitor document contains instructions that attempt to steer the agent	High	The agent ingests untrusted external documents, so Azure AI Content Safety Prompt Shields (jailbreak and indirect-attack detection) are enabled on all document-derived content; extracted text is treated as data, not instructions (14.4).
Mishandling of competitor-confidential information	Medium	Data minimisation and retention limits (Section 7.4, 10.3); access restricted by identity (10.1); documents stored only as long as needed for the comparison and audit trail.
Automation bias — the consultant trusts the draft without checking	Medium	The review surface shows source figures and citations, not just the conclusion (14.6); the draft is explicitly labelled AI-generated and not yet sent.
Unfair performance across quote formats / smaller builders	Low	No decisions are made about people, so demographic fairness risk is minimal; quality fairness across document formats is addressed by an evaluation set spanning competitor templates (15.2).

14.3 Mapping to the Six Responsible AI Principles

Each principle is mapped to the concrete control in this design that satisfies it. Cross-references point to the section that holds the detail.

Principle	How this design addresses it
Fairness	The agent performs line-item price matching, not decisioning about individuals, so there is no demographic fairness exposure. The relevant fairness risk is uneven extraction quality across competitor document formats; this is measured on a representative evaluation set (15.2) and tracked as a quality metric (15.3).
Reliability & Safety	Outputs are grounded to the Henley pricing source of truth (14.5); comparison logic is deterministic where the data allows; automated evaluation gates (15.4) guard against regressions; the reconciliation sweep and exception model (Section 8) prevent silent failure; human review (14.6) precedes any customer-facing use.
Privacy & Security	Customer and competitor data are minimised and retained only as needed (7.4, 10.3). Prompts and completions are not used to train foundation models (Azure OpenAI / Azure AI Foundry data-handling commitment). Identity, secrets, network isolation and audit are covered in Section 10.
Inclusiveness	Generated email drafts use plain, accessible language and a consistent readable structure so the output is usable by a broad customer base and by consultants of varying experience.
Transparency	Drafts are clearly marked as AI-generated and unsent; the review surface exposes the source figures and citations behind each comparison; the agent's limitations and intended use are documented here and disclosed to internal users.

Principle	How this design addresses it
Accountability	A named human (the reviewing consultant) approves or rejects every external-facing output; a named RAI owner signs off this section; the Process Ledger (Section 7) provides an end-to-end audit trail of who reviewed and released what.

14.4 Content Safety and Guardrails

Because the agent ingests documents supplied from outside Henley, content safety is configured defensively at two points: on model inputs and on model outputs. Azure AI Content Safety / Azure AI Foundry content filters are enabled for the hate, sexual, violence and self-harm categories at the platform default severity thresholds, tuned during testing. Prompt Shields are enabled to detect jailbreak attempts and indirect (document-embedded) prompt-injection attacks — a material threat for this use case, since competitor quotes are untrusted input. Extracted document text is handled as data to be compared, never as instructions to be followed. Groundedness detection and, where licensed, protected-material detection are applied to generated output. A small custom blocklist covers terms Henley does not want appearing in any customer-facing draft. All content-safety triggers are logged to the telemetry described in Section 11 and routed through the exception model in Section 8.

14.5 Grounding and Hallucination Mitigation

The defining control on accuracy is that the model reasons over evidence rather than from memory. Henley prices, inclusions and specifications are retrieved from the authoritative pricing and document store (Section 5.6, 6.2) and supplied to the comparison step as grounding context; the model's job is to align and explain, not to recall or invent figures. Numeric values that appear in the draft are taken from the pricing store and validated against it, not free-generated by the model. Where the evidence is missing, ambiguous or low-confidence, the agent follows the Section 8 rule and escalates to human review instead of filling the gap. Groundedness is an explicit, measured acceptance metric (15.3).

14.6 Human Oversight as a Responsible AI Control

The human-in-the-loop step described in Section 5.4 is, in Responsible AI terms, the accountable-human control for the entire solution. No comparison reaches a customer without a Henley consultant reviewing the draft, the matched line items and the underlying source figures, and choosing to edit, approve or reject it. The review surface is deliberately designed to counter automation bias: it presents the evidence and the citations alongside the conclusion, so the reviewer is checking the agent's working rather than rubber-stamping its answer. Approval, edits and rejections are all recorded in the Process Ledger, giving a defensible record of human accountability for every output.

15. AI Evaluation, Test Strategy and Acceptance

15.1 Evaluation Approach

This section defines the evaluation methodology, metrics and acceptance thresholds as design commitments. They constrain the build and set the bar the solution must clear before release. The executed evidence — the evaluation report, the test logs and the signed UAT record — is produced during delivery and is referenced here rather than reproduced; those artefacts are the evidence that

satisfies the service-validation control at go-live. Evaluation is implemented using the Azure AI Foundry evaluation tooling, run both offline against a curated dataset and as an automated gate in the deployment pipeline (15.4).

15.2 Evaluation Dataset (Golden Set)

A curated golden set of representative competitor quotes is the foundation of evaluation. It is assembled with Henley's input to span the document formats, builders and price structures the agent will meet in production, and is deliberately weighted towards the edge cases that break naive extraction: scanned PDFs, multi-page quotes, bundled inclusions, promotional pricing and incomplete quotes. Each item is paired with an expected comparison output agreed with a Henley subject-matter expert. The set is version-controlled and grows as production surfaces new patterns; a target of at least [40–60] curated cases is proposed for the initial baseline, to be confirmed during Design.

15.3 Evaluation Metrics and Thresholds

The agent is evaluated on extraction quality, comparison correctness, the quality of the generated email and its safety behaviour. Thresholds below are the proposed acceptance bar and are confirmed with Henley during UAT planning.

Metric	Definition	Proposed target
Extraction accuracy	Field-level correctness of values read from the competitor quote against the golden set	≥ 95% on key fields
Comparison correctness	Correct matching of competitor line items to Henley equivalents and correct variance direction	≥ 95%
Groundedness	Proportion of stated figures and claims traceable to the pricing source of truth (no unsupported numbers)	≥ 99%
Hallucination rate	Generated statements not supported by evidence	≤ 1%
Email quality	Relevance, coherence and tone of the draft, scored against a rubric by reviewers	≥ 4 / 5 mean
Refusal appropriateness	Correctly escalating or refusing on insufficient-evidence and unsafe-input cases	≥ 98%
Latency (per request)	End-to-end processing time, intake to draft ready for review	Within NFR target (Section 9)

15.4 Automated Evaluation in the Pipeline

The golden-set evaluation runs automatically on every change to a prompt, a model version or the comparison logic, and acts as a release gate: a change that drops any metric below threshold does not promote. Model and prompt versions are pinned and recorded so a regression can be traced to a specific change. This turns evaluation from a one-off sign-off activity into a standing regression guard, and produces the metric history that evidences ongoing reliability after go-live.

15.5 User Acceptance Testing

UAT validates the solution against real Henley working practice with the people who will use it. It is scoped to confirm that, for representative live-like quotes, the agent produces drafts a consultant

accepts with little or no edit, that the review-and-release flow fits the sales process, and that exceptions behave as designed. Participants are nominated Henley sales consultants and a business owner, supported by the Fusion5 delivery team.

Entry criteria: the solution is deployed to the UAT environment, the golden-set evaluation passes its thresholds, and the test scenarios and data are agreed. Exit criteria: all priority-1 and priority-2 defects are resolved or accepted with a workaround, the acceptance criteria in 15.6 are met, and the Henley business owner signs off. The detailed UAT plan, scenario scripts, defect log and signed acceptance record are delivery artefacts referenced by this design.

15.6 Acceptance Criteria

The solution is accepted for production when the following are demonstrably true. These extend the per-stage Definition of Done (Section 4.2, 6.4) with measurable, signed-off thresholds and are the contract for “done”.

Acceptance criterion	Basis / verification
Golden-set evaluation meets all thresholds in 15.3	Automated evaluation report
UAT scenarios pass with consultant acceptance of drafts	Signed UAT record
No open priority-1 or priority-2 defects	Defect log triage
Responsible AI section (14) reviewed and signed off	RAI owner sign-off
Security and compliance controls (Section 10) verified	Security review / WAF assessment
Monitoring, alerting and run book in place (Sections 11, 17)	Operational readiness review
Henley business owner authorises go-live	Production sign-off

16. Deployment and Release Approach

This section sets the deployment and release strategy at the level a design needs to fix. The executable detail — pipeline definitions, environment-specific configuration, the cutover runbook and the go-live checklist — is produced during build and recorded in the delivery artefacts referenced below.

16.1 Deployment Strategy

The workload is deployed onto the Azure landing zone designed for Henley by the Fusion5 Cloud Centre of Excellence, derived from the proven reusable substrate template and configured to this workload's shape (an event-driven pipeline with a Foundry agent and a Teams human-in-the-loop step). All infrastructure is defined as code (Bicep / Azure Verified Modules, Terraform where the CCoE standardises on it) so environments are reproducible and auditable. Application components — Logic Apps, Durable Functions, the Foundry agent definition and prompts — are deployed by pipeline, never by hand. Promotion runs through separate dev, test, UAT and production environments (Section 3.5), each isolated and configured from code; the landing-zone build is a hard prerequisite for the production deployment.

16.2 CI/CD Pipeline

Build, infrastructure and application changes flow through a CI/CD pipeline (GitHub Actions or Azure DevOps, per the customer standard) authenticating to Azure with workload-identity federation (OIDC) rather than stored secrets. The pipeline builds, runs automated tests and the golden-set evaluation gate (Section 15.4), and promotes only on success. Promotion to UAT and production is gated by explicit human approval. Infrastructure and application pipelines are separated so the landing zone and the workload can be governed and released independently.

16.3 Release Management, Cutover and Rollback

Go-live follows a controlled cutover with a documented checklist and a defined rollback path. A piloted rollout is recommended — a limited set of consultants and quote volumes first — so behaviour and cost are confirmed against real traffic before broad enablement, consistent with the cost-governance controls in Section 12. Because the agent is stateless between requests and the durable state lives in the Process Ledger, rollback is primarily a matter of reverting the deployed version and re-pointing configuration; in-flight items are protected by the idempotency and reconciliation controls in Sections 6.5 and 8.5. A hypercare period follows go-live, with the delivery team on heightened support and the evaluation and telemetry metrics watched closely. The cutover runbook, rollback procedure and go-live checklist are delivery artefacts.

17. Operational Handover and Support Model

This section commits to the operating model, responsibilities and service levels for the solution in production. The full operational run book, contact lists and on-call arrangements are go-live artefacts referenced here; they are deliberately not frozen into this design-stage document where they would quickly go stale.

17.1 Target Operating Model

The proposed model is a Fusion5-supported service with defined Henley responsibilities, confirmed commercially before go-live. Day-to-day use and first-line triage of business questions sit with Henley; platform, application and AI support sit with Fusion5. Support is tiered: L1 handles user questions and known issues from the run book; L2 handles application and integration faults; L3 handles platform, model and deep engineering issues. The boundary between Henley and Fusion5 responsibilities is set out in the RACI below.

17.2 Responsibilities (RACI)

Activity	Henley	Fusion5 L1/L2	Fusion5 L3	RAI Owner
Day-to-day use and draft review	R/A	I	-	-
Maintaining the pricing source of truth	R/A	I	-	-
Incident response and fault fixing	I	R/A	C	-
Model / prompt changes and re-evaluation	C	C	R/A	C

Activity	Henley	Fusion5 L1/L2	Fusion5 L3	RAI Owner
Content-safety and RAI review	C	I	C	R/A
Cost monitoring and optimisation	I	R	A	-

17.3 Service Levels

Indicative service-level targets are proposed below and confirmed in the support agreement. Availability is set against the non-functional requirements in Section 9; because the agent is an assistive, non-real-time process, the targets are pragmatic rather than mission-critical.

Severity	Target response	Target resolution / workaround
P1 — service down / no drafts produced	1 business hour	Same business day
P2 — degraded quality or partial failure	4 business hours	2 business days
P3 — minor issue / question	1 business day	Next release or as scheduled

17.4 Run Book and Knowledge Transfer

A operational run book is produced at go-live covering routine operations, the alert-to-action playbook for each alert in Section 11.3, the exception-handling procedures from Section 8, and escalation paths. Knowledge transfer to the support team and to Henley super-users is delivered before handover. The run book and KT materials are delivery artefacts referenced by this design.

18. Business Continuity and Disaster Recovery

Resilience targets are design decisions because they drive region, redundancy and SKU choices, so they are fixed here; the executed DR test and its evidence are a delivery artefact.

18.1 Availability and Resilience Design

The workload is deployed to the Australia East region on the landing zone described in Section 16.1, using availability-zone-redundant platform services where offered. Resilience is reinforced by the design itself: the agent is stateless between requests, durable state and progress live in the Process Ledger (Section 7), and the idempotency and reconciliation controls (Sections 6.5, 8.5) mean an interrupted request can be safely retried or recovered without duplication or loss. Transient failures are absorbed by retry-with-backoff rather than escalated.

18.2 Recovery Objectives

Because the agent is an assistive, non-real-time process, recovery objectives are pragmatic. Targets below are proposed and confirmed during Design.

Component	RPO (data loss tolerance)	RTO (time to restore)
Process Ledger / state store	≤ 1 hour	≤ 4 hours
Pricing and document store	≤ 24 hours (source maintained externally)	≤ 4 hours

Component	RPO (data loss tolerance)	RTO (time to restore)
Orchestration / compute	Stateless — no data RPO	≤ 4 hours (redploy from IaC)

18.3 Backup and Recovery

The Process Ledger and the document/pricing store are backed up on a schedule consistent with the RPO targets above, using the platform-native backup of the underlying data services. Because all infrastructure is defined as code (Section 16.1), the compute and orchestration tier is recovered by redeployment rather than restore, which keeps the RTO achievable. Recovery procedures are documented in the run book (Section 17.4).

18.4 DR Testing

A disaster-recovery test is performed before go-live and on an agreed periodic basis thereafter, exercising restore of the state and document stores and redeployment of the workload from code, and confirming the recovery objectives are met. The DR test plan and results are a delivery artefact.

19. Requirements Traceability and Control Mapping

This section makes the design auditable in two directions. The first table traces the principal requirements to the design that satisfies them and the means by which each is verified. The second maps the design and its evidence to the Microsoft AI Platform specialisation (Module B) controls, so an assessor can move from a control directly to where it is addressed and to the artefact that evidences it.

19.1 Requirements Traceability Matrix

Requirement (FRD / NFR)	Design reference	Verification
Compare competitor quote to Henley pricing	Sections 5.2–5.3, 6.2	Comparison-correctness evaluation (15.3)
Produce customer-ready draft email	Section 5.5	Email-quality evaluation; UAT (15.5)
Human review before any customer use	Sections 5.4, 14.6	UAT; Process Ledger audit (Section 7)
No invented or unsupported figures	Sections 8.1, 14.5	Groundedness / hallucination metrics (15.3)
Safe handling of untrusted documents	Section 14.4	Prompt-shield / content-safety testing
Security, identity and data protection (NFR)	Section 10	Security review / WAF assessment
Availability and recovery (NFR)	Sections 9, 18	DR test (18.4)
Cost within agreed envelope (NFR)	Section 12	Cost monitoring (12.6, 17.3)

19.2 Module B Control Evidence Map

This SDD is the design evidence for the Solution Design control. The deployment, validation and post-deployment controls are satisfied by the executed delivery artefacts this document references, produced at and after go-live. The map below states, for each control family, where it is addressed and the evidence artefact that closes it — a design-stage SDD cannot itself satisfy the production controls.

Module B control	Addressed in	Evidence artefact
Assess / Portfolio Assessment	Sections 1–2 (context, problem, scope)	AI Envisioning Workshop; Discovery Synthesis
Solution Design	This SDD — Sections 3–19	This document (baselined)
Well-Architected Review	Section 3.6	Azure Well-Architected Review export
Proof of Concept	Sections 4–5, 15	Evaluation report on golden set
Deployment	Section 16	Release record; production deployment
Service Validation	Section 15	Signed UAT record; evaluation report
Post-deployment Operations	Sections 11, 17, 18	Run book; support agreement; DR test

Appendix A — Glossary

Term	Definition
Ambiguity flag	A request for human confirmation raised when a competitor inclusion lacks the detail needed for a reliable comparison.
Competitor quote	A formal pricing document from a competing builder (e.g. Carlisle, Metricon) for a specific home design.
Confidence score	A HIGH/MEDIUM/LOW indicator on each comparison expressing the agent's certainty that two items are genuinely comparable.
Definition of done	The verified end state: a DRAFT comparison delivered to the consultant, every figure traceable, reflected in the ledger.
FinOps	Cloud financial operations (Section 12). Not Dynamics 365 Finance & Operations — the term clashes; this document means the former.
Hallucination	A fabricated cost, quantity or inclusion not verifiable from the source documents; structurally prevented (Section 5.3).
Henley advantage	An inclusion Henley provides as standard that is absent from, or lesser in, the competitor's quote.
Henley pricing sheet	The internal spreadsheet of inclusion costs, upgrade prices and regional variants; the pricing system of record.
Idempotency	The property that a stage can run twice without creating a duplicate comparison or email.
Orphan	A request that has entered the system but has no owner, next action or visibility; the failure the design exists to prevent.

Term	Definition
Process ledger	The agent's own record of every request and its state; the single source of truth (Section 7).
Session isolation	Each request processed fully independently, with no data or context carried over between runs.
SME	Subject-matter expert — Adrian Serratore; authority on pricing, ambiguity rulings and the email format.
Straight-through rate	The share of requests completed with no human intervention; the primary cost and efficiency lever.

Appendix B — Open Questions

The items below must be closed in the Design phase before this SDD is baselined. The first six are carried from the FRD; the remainder arise from this design.

#	Question	Owner
OI-01	The definitive list of state variants in the Henley pricing data (VIC, NSW, QLD, nationwide — any others?).	Adrian Serratore
OI-02	Accepted input formats for competitor quotes (PDF only, or also image/Word/email body?).	Sales Team Lead
OI-03	Whether the pricing data is a single multi-tab workbook or several files per state.	Adrian Serratore
OI-04	The standard category headings used to group line items in the comparison.	Adrian Serratore
OI-05	Email tone and signature block — does the draft pre-fill the consultant's name and contact?	Sales Team Lead
OI-06	How to handle competitors beyond Carlisle and Metricon (e.g. Metricon, Sheridan, Brighton ranges).	Adrian Serratore
OI-07	Document and data retention period, aligned to Henley's data-handling and privacy policy.	Henley IT (D. Pike)
OI-08	The confidence threshold for automatic acceptance, and the human-review SLA.	Fusion5 / Adrian
OI-09	Network posture and any DLP requirements for the Azure environment.	Henley IT (D. Pike)
OI-10	Whether a low-volume Consumption intake should replace the Logic Apps Standard plan.	Solution Architect
OI-11	Run and export the two-pillar (Reliability and Security) WAR-for-AI assessment and agree the remediation list with the platform owner.	Fusion5 / D. Pike